

SOMMAIRE

Etape 1: initialisation des tableaux tabjouer & tabordi.....	2
Etape 2: On place nos 5 pions: tabjouer.....	2
Etape 3: on place les 5 pions de l'ordinateurs : tabordi.....	3
Etape 5: Creer la procedure affichagetabCache.....	5
Etape 6 : Le joueur cherche un pion.....	6
Etape 7 : L'ordinateur cherche un pion.....	8
Etape 8 : fin de partie.....	9
CONCLUSION.....	10

Etape 1: initialisation des tableaux tabjouer & tabordi

Déclarer un table de 5 lignes et 5 colonnes, et attribuer à chaque ligne et chaque colonne la valeur 0

```
public class etape1 {
    public static void main(String[] args) {

        // ÉTAPE 1 : Initialisation des tableaux 5x5
        int[][] tabJoueur = new int[5][5];
        int[][] tabOrdi = new int[5][5];

        // Attribuer chaque ligne et chaque colonne la valeur 0
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                tabJoueur[i][j] = 0;
                tabOrdi[i][j] = 0;
            }
        }
    }
}
```

Etape 2: On place nos 5 pions: tabjouer

2.1 et 2.2:

Demander la ligne et Demander la colonne

```
// 2.1 Demander la ligne
System.out.print("Choisis la ligne (0 à 4) : ");
int ligne = sc.nextInt();

// 2.2 Demander la colonne
System.out.print("Choisis la colonne (0 à 4) : ");
int colonne = sc.nextInt();
```

2.3: afficher les deux tableaux pour vérifier les placements

```
if (ligne < 0 || ligne >= 5 || colonne < 0 || colonne >= 5) {
    System.out.println("Erreur: position en dehors du tableau !");
} else if (tabJoueur[ligne][colonne] == 1) {
```

```

        System.out.println("Cette case à déjà un pion !");
    } else {
        tabJoueur[ligne][colonne] = 1;
        pionsPlaces++;
        System.out.println("Pion place en (" + ligne + ", " + colonne + ")");
    }
}

```

2.4: on répète étape 2.1 à 2.3 tant que le joueur n'a pas placé tous les pions

```
while (pionsPlaces < nbPions) {
```

Etape 3: on place les 5 pions de l'ordinateurs : tabordi

3.1: Ligne aléatoire

```
int ligneOrdi = (int)(Math.random() * 5);
```

3.2: Colonne aléatoire

```
int colonneOrdi = (int)(Math.random() * 5);
```

3.3: Vérification et placement

```

if (tabOrdi[ligneOrdi][colonneOrdi] == 0) {
    tabOrdi[ligneOrdi][colonneOrdi] = 1;
    pionsOrdiPlaces++;
}
}

```

3.4: on répète étape 3.1 à 3.3 tant que l'ordinateur n'a pas placé tous les pions

```
while (pionsOrdiPlaces < nbPions) {
```

Voici les resultat de tout ces etape,

Choisis la ligne (0 à 4) : 2

Choisis la colonne (0 à 4) : 3

✓ Pion placé en (2, 3)

Choisis la ligne (0 à 4) : 2

Choisis la colonne (0 à 4) : 2

⚠ Cette case contient déjà un pion !

Etape 4: Créer la procedure affichagetab

4.1: Creation de la procedure affichagetab

```
static void affichagetab(int tab[], int nbcase) {
```

4.2: Afficher les numeros de colonnes

```
System.out.print(" ");  
for (int i = 1; i <= nbcase; i++) {  
    System.out.print(i + " ");  
}  
System.out.println();
```

4.3: Afficher le numero de ligne et les cases

```
System.out.print((i + 1) + " ");  
for (int j = 0; j < nbcase; j++) {  
    if (tab[i][j] == 1) {  
        System.out.print("o ");  
    } else {  
        System.out.print("~ ");  
    }  
}
```

4.4: Repeter pour toutes les lignes

```
for (int i = 0; i < nbcase; i++) {
```

Voici les résultat après tout cela, après avoir choisir tout les nombre,

```
 1 2 3 4 5  
1 o ~ ~ ~ ~  
2 ~ ~ ~ ~ ~  
3 o ~ ~ ~ o  
4 ~ o ~ ~ ~  
5 ~ ~ ~ ~ o
```

Etape 5: Creer la procedure affichagetabCache

```
static void affichagetabCache(int tab[], int nbcase) {  
    // Afficher les numeros de colonnes  
    System.out.print(" ");  
    for (int i = 1; i <= nbcase; i++) {  
        System.out.print(i + " ");  
    }  
    System.out.println();  
    // Pour chaque ligne  
    for (int i = 0; i < nbcase; i++) {  
        System.out.print((i + 1) + " ");  
        for (int j = 0; j < nbcase; j++) {  
            if (tab[i][j] == 0) {  
                System.out.print("? ");  
            }  
            else if (tab[i][j] == 1) {  
                System.out.print("? ");  
            }  
            else if (tab[i][j] == 2) {  
                System.out.print("o ");  
            }  
            else if (tab[i][j] == 3) {  
                System.out.print("x ");  
            }  
        }  
        System.out.println();  
    }  
}
```

"Dans l'étape 5, j'ai créé une nouvelle procédure qui s'appelle **affichagetabCache**. Cette procédure sert à afficher la grille de l'adversaire mais en cachant les bateaux qu'on n'a pas encore trouvés. Quand la case vaut 0 ou 1, j'affiche un point d'interrogation pour cacher l'information. Quand la case vaut 2, ça veut dire qu'on a touché un bateau donc j'affiche 'o'. Et quand la case vaut 3, ça veut dire qu'on a tiré mais raté donc j'affiche 'x'. Comme ça, le joueur ne peut pas tricher en voyant où sont les bateaux ennemis."

Et voici le resultat,

```
Choisis la ligne (0 a 4) : 2
Choisis la colonne (0 a 4) : 3
Cette case contient deja un pion !
Choisis la ligne (0 a 4) : 4
Choisis la colonne (0 a 4) : 1
Pion place en (4, 1)
Grille du joueur :
1 2 3 4 5
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ o o ~
4 ~ ~ o o ~
5 ~ o ~ ~ ~
Grille de l'ordinateur :
1 2 3 4 5
1 o ~ ~ ~ ~
2 o ~ ~ o ~
3 ~ ~ ~ ~ o
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ o
```

Etape 6 : Le joueur cherche un pion

6.1: Demandez a l'utilisateur a quelle ligne il veut decouvrir une case

```
System.out.print("Choisis la ligne (0 a 4) : ");
int ligneTir = sc.nextInt();
```

6.2 : Demandez a l'utilisateur a quelle colonne il veut decouvrir une case

```
System.out.print("Choisis la colonne (0 a 4) : ");
int colonneTir = sc.nextInt();
```

6.3 : Afficher "Tir en cours" et laisser une attente de 2 secondes pour "simuler" le tir

```
System.out.println("Tir en cours...");
try {
    Thread.sleep(2000);
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

6.4 Afficher le resultat

Si la case etait sans pion (valeur 0) : il faut afficher rate !

```
if (tabOrdi[ligneTir][colonneTir] == 0) {
    System.out.println("Rate !");
    tabOrdi[ligneTir][colonneTir] = 3; // 3 = case rate
}
```

Si la case était avec pion (valeur 1) : afficher Touche ! + incrementer la variable nbPionTrouveJoueur

```
else if (tabOrdi[ligneTir][colonneTir] == 1) {  
    System.out.println("Touche !");  
    tabOrdi[ligneTir][colonneTir] = 2; // 2 = pion touche  
    nbPionTrouveJoueur++;  
}
```

Toutes autres valeurs dans la case cela signifie que le joueur avait déjà découvert cette case : afficher Tir à blanc !

```
else {  
    System.out.println("Tir à blanc !");  
}
```

6.5 Afficher le tableau ordi : Faites juste un appel à la procédure créée en étape 5

```
System.out.println("\nGrille de l'ordinateur :");  
afficheTabCache(tabOrdi, 5);  
sc.close();  
}
```

"Dans l'étape 6, c'est le tour du joueur pour tirer sur un bateau. D'abord, j'affiche 'A vous de jouer'. Ensuite, je demande au joueur de choisir une ligne avec un Scanner, puis je lui demande de choisir une colonne. Après, j'affiche 'Tir en cours' et j'utilise Thread.sleep pour faire une pause de 2 secondes, ça rend le jeu plus réaliste. Ensuite, je regarde la valeur de la case : si c'est 0, j'affiche 'Raté' et je mets la valeur à 3. Si c'est 1, j'affiche 'Touché', je mets la valeur à 2 et j'ajoute 1 au compteur de bateaux trouvés. Si c'est déjà 2 ou 3, j'affiche 'Tir à blanc' parce que le joueur a déjà tiré là. À la fin, j'appelle la procédure de l'étape 5 pour afficher la grille avec les cases cachées."

Etape 7 : L'ordinateur cherche un pion

7.0 Afficher "Tour de l'ordinateur"

```
System.out.println("\n=====");
System.out.println("Tour de l'ordinateur");
System.out.println("=====");
boolean tirValide = false;
while (!tirValide) {
```

7.1 Attribuer une ligne aleatoirement (entre 1 et 5)

```
int ligneTirOrdi = (int)(Math.random() * 5);
```

7.2 Attribuer une colonne aleatoirement (entre 1 et 5)

```
int colonneTirOrdi = (int)(Math.random() * 5);
```

7.3 Attention l'ordinateur ne doit pas choisir une case qui a ete decouverte (valeur autre que 0 ou 1)

```
if (tabJoueur[ligneTirOrdi][colonneTirOrdi] == 0 || tabJoueur[ligneTirOrdi][colonneTirOrdi] == 1) {
    tirValide = true;
```

7.4 Afficher "Tir en cours" et laisser une attente de 2 secondes pour "simuler" le tir

```
System.out.println("Tir en cours...");
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

7.5 Afficher le resultat

Si la case etait sans pion (valeur 0) : afficher Rate !

```
if (tabJoueur[ligneTirOrdi][colonneTirOrdi] == 0) {
    System.out.println("L'ordinateur a rate !");
    tabJoueur[ligneTirOrdi][colonneTirOrdi] = 3; // 3 = case rate
}
```

Si la case etait avec pion (valeur 1) : afficher Touche ! + incrementer la variable nbPionTrouveOrdi

```
else if (tabJoueur[ligneTirOrdi][colonneTirOrdi] == 1) {
    System.out.println("L'ordinateur a touche !");
    tabJoueur[ligneTirOrdi][colonneTirOrdi] = 2; // 2 = pion touche
    nbPionTrouveOrdi++;
    System.out.println("L'ordinateur a trouve " + nbPionTrouveOrdi + "/5 bateaux");
}
}
```

7.6 Afficher le tableau joueur : Faites juste un appel a la procedure cree en etape 5

```
System.out.println("\nVotre grille :");
afficheTabCache(tabJoueur, 5);
}
```


Etape 8 : fin de partie

Etape 8.2 : Afficher le vainqueur

```
System.out.println("\n=====");
System.out.println("      FIN DE PARTIE");
System.out.println("=====");
if (nbPionTrouveJoueur >= 5) {
    System.out.println("VICTOIRE ! Vous avez gagne !");
} else {
    System.out.println("DEFAITE ! L'ordinateur a gagne !");
}
```

Etape 8.3 : Demandez de refaire une partie

```
System.out.print("\nVoulez-vous refaire une partie ? (oui/non) : ");
String reponse = sc.next();
if (reponse.equalsIgnoreCase("oui")) {
    System.out.println("Relancez le programme pour rejouer !");
} else {
    System.out.println("Merci d'avoir joue !");
}
sc.close();
}
```

CONCLUSION

Ce tp de bataille navale, j'ai réussi à créer un jeu complet en Java en suivant toutes les étapes du TP. J'ai d'abord appris à créer et initialiser des tableaux à deux dimensions pour représenter les grilles de jeu. Ensuite, j'ai utilisé le Scanner pour permettre au joueur de placer ses bateaux et de tirer sur l'adversaire, et j'ai utilisé Math.random() pour que l'ordinateur joue automatiquement. J'ai également créé deux procédures d'affichage : une pour montrer toute la grille, et une autre pour cacher les bateaux non découverts, ce qui rend le jeu plus réaliste. Grâce aux boucles while, le jeu continue jusqu'à ce qu'un des deux joueurs trouve les 5 bateaux adverses. Ce projet m'a permis de bien comprendre les tableaux, les boucles, les conditions if/else, et l'utilisation de procédures en Java. C'était intéressant de voir comment toutes ces notions s'assemblent pour créer un jeu fonctionnel.